Software Requirements Specification

For

ML Based Crop Disease Detection

Detection of Crop Disease Using CNN and RAG-Enabled Decision Support

Prepared by: Sushanth

USN: 4JK25MC106

Project Guide:Amita Roshan Vakil

Course: Master of Computer Applications (MCA)

Date: 05/10/2026

Academic Year: 2026–2027

Chapter / Section	Title	Page No.
1	INTRODUCTION……..	3
1.1	Purpose……...	3
1.2	Intended Audience…….	3
1.3	Scope……...	3
1.4	References……...	4
2	OVERALL DESCRIPTION……...	4
2.1	Product Perspective……	4
2.2	Product Functions……...	5
2.3	User Classes and Characteristics……	5
2.4	Operating Environment……..	5
2.5	Design and Implementation Constraints…….	5
2.6	Assumptions and Dependencies……	6
3	SPECIFIC REQUIREMENTS……...	6
3.1	Data Requirements……..	6
3.2	Functional Requirements……	7
3.3	Performance and ML Quality Requirements……..	7
3.4	External Interface Requirements……	8
3.5	Security and Safety Requirements……..	8
3.6	Ml model Requirement……...	8
3.7	RAG and Decision support Requirement……	9
4	APPENDICES……	9
4.1	Glossary……...	10
4.2	Sample Data Schemas / EDA……..	10
4.3	Future Enhancements……..	11

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, machine-learning, interface, performance, security, and quality requirements of the "ML Based Crop Disease Detection" system. The system is intended to assist farmers and agricultural workers in identifying crop diseases from leaf images and obtaining grounded decision-support information.

1.2 Intended Audience

- Software developers and ML engineers responsible for implementation and maintenance.
- System administrators managing users, scan records and advisor information.
- Farmers and agricultural workers using the web application.
- Agricultural advisors who provide professional consultation.

1.3 Scope

- The system accepts crop leaf images through a responsive web interface and performs image preprocessing and Region of Interest (ROI) extraction before disease classification.
- The machine-learning component uses the PlantVillage dataset and the project's defined crop and disease classes. The image is classified using a VGG16-based CNN with a Spatial Attention mechanism.
- The system provides a confidence score for each prediction and applies a predefined confidence threshold to identify uncertain classifications rather than presenting them as definite diagnoses.
- The RAG-based decision-support component retrieves relevant information from a curated agricultural knowledge base and provides disease-specific management, precaution, and prevention information.
- The application can generate a downloadable PDF report containing the detected disease, confidence score, and relevant retrieved management information.
- The system supports an agricultural consultation module that provides advisor information and available appointment slots, allowing users to access and book agricultural consultation.
- The system is intended as a decision-support tool and does not replace the advice or diagnosis of qualified agricultural professionals.

1.4 References

1. Albattah, W., Nawaz, M., Javed, A., Masood, M., & Albahli, S. (2022). A novel deep learning method for detection and classification of plant diseases. *Complex & Intelligent Systems, 8*, 507–524. https://doi.org/10.1007/s40747-021-00536-1
2. Kochhar, A., Narang, G., Patel, S., Kaur, H., Singla, C., Pateriya, B., & Galdelli, A. (2026). A three-tier deep learning framework with mobile application integration for multi-crop disease diagnosis. *Discover Artificial Intelligence, 6*, 358. https://doi.org/10.1007/s44163-026-01062-0
3. Ngugi, H. N., Ezugwu, A. E., Akinyelu, A. A., & Abualigah, L. (2024). Revolutionizing crop disease detection with computational deep learning: A comprehensive review. *Environmental Monitoring and Assessment, 196*, 302. https://doi.org/10.1007/s10661-024-12454-z
4. Santoshi, C., Mandala, S. K., & Raju, L. R. (2026). LDDHybridNet: An ROI-aware CNN–LSTM hybrid framework for accurate and early leaf disease detection in precision agriculture. *Scientific Reports, 16*, 20284. https://doi.org/10.1038/s41598-026-50398-1
5. Tuteja, G., Al-Yarimi, F. A. M., Ikram, A., Gupta, R., Rehman, A. U., Singh, J., Delgado Noya, I., & Dzul Lopez, L. A. (2026). CNNAttLSTM: An attention-enhanced CNN–LSTM architecture for high precision jackfruit leaf disease classification. *Frontiers in Plant Science, 16*, 1720471. https://doi.org/10.3389/fpls.2025.1720471

6. Patil, M. A., & Manohar, M. (2023). Plant leaf disease classification using optimal tuned hybrid LSTM-CNN model. *SN Computer Science, 4*, 710. https://doi.org/10.1007/s42979-023-02245-7
7. *AgroVisionNet*. (2026). [Scientific Reports, 16, 2479]. https://doi.org/10.1038/s41598-025-32384-1
8. Alirezazadeh, P., Schirrmann, M., & Stolzenburg, F. (2023). Improving deep learning-based plant disease classification with attention mechanism. *Gesunde Pflanzen, 75*, 49–59. https://doi.org/10.1007/s10343-022-00796-y
9. Muthu, B., & Cherubini, C. (2026). A novel approach for crop disease detection using transfer-learning-based deep convolutional neural networks. *Neural Computing and Applications, 38*, 500. https://doi.org/10.1007/s00521-026-12216-7
10. Dai, G., Fan, J., Tian, Z., Wang, C., et al. (2023). PPLC-Net: Neural network-based plant disease identification model supported by weather data augmentation and multi-level attention mechanism. *Journal of King Saud University–Computer and Information Sciences, 35*, 101555. https://doi.org/10.1016/j.jksuci.2023.101555
11. Wang, X., & Liu, J. (2026). Enhancing plant disease detection through multi-modal integration of visual and textual data. *Plant Methods, 22*, 30. https://doi.org/10.1186/s13007-026-01521-w

2. OVERALL DESCRIPTION

2.1 Product Perspective

The system is a web-based ML decision-support application consisting of a user interface, Flask application backend, image-processing pipeline, trained CNN model, RAG/knowledge layer, MongoDB database, PDF reporting module and consultation module.

Component	**Role**
Web Interface	Image upload, authentication, scan results, history, reports and advisor consultation.
Flask Backend	Application routing, request handling, model integration and database operations.
Preprocessing & ROI	Resize, normalization and HSV-based ROI extraction to reduce background influence.
VGG16 + Spatial Attention	Extract visual features and emphasize informative spatial regions.
Classifier	Predict the defined crop/disease class and confidence score.
RAG Knowledge Layer	Retrieve relevant agricultural information for the predicted disease.
Gemini API	Generate disease-specific decision-support guidance from retrieved/contextual information.
MongoDB	Store user information, scan history, advisor information and appointment data.
ReportLab	Generate downloadable PDF diagnostic reports.

2.2 Product Functions

- User Management: User registration, login, authentication, and account management.
- Crop Leaf Image Processing: Upload and validation of crop leaf images, followed by preprocessing and Region of Interest (ROI) extraction.
- ML-Based Disease Detection: VGG16-based CNN classification with a spatial attention mechanism to identify the supported crop disease classes.
- Confidence-Aware Prediction: Generation of a confidence score and identification of low-confidence/uncertain predictions using a defined threshold.
- RAG-Based Decision Support: Retrieval of disease-specific information from the curated agricultural knowledge base and generation of relevant decision-support guidance.
- Disease Management Information: Display of symptoms, precautions, preventive measures, and management/solution information for the detected disease.
- Diagnostic Report Generation: Generation and downloading of a PDF report containing the disease detection result, confidence information, and decision-support information.
- Scan History: Storage and viewing of previous crop disease detection results.
- Agricultural Advisor Support: Display of agricultural advisor information and available expertise.
- Appointment Booking: Viewing available advisor slots and booking appointments for consultation.

2.3 User Classes and Characteristics

User Class	Characteristics / Responsibilities
Farmer / Agricultural Worker	Uploads leaf images, views predictions, reads guidance, downloads reports and may book an advisor.
Agricultural Advisor	Provides professional consultation through available profile information and appointment slots.
Developer / ML Engineer	Maintains preprocessing, trained model, RAG pipeline, APIs and application components.

2.4 Operating Environment

- OS: Windows 10/11, macOS or Linux.
- Programming: Python 3.9+, HTML5, CSS3 and JavaScript (ES6+).
- Frameworks/libraries: Flask, PyTorch, TorchVision, OpenCV, ReportLab and Leaflet.js.
- Database: MongoDB local or MongoDB Atlas.
- Generative AI/retrieval: Google Gemini API with curated RAG knowledge base.
- Development environment: VS Code or PyCharm.
- Hardware: Intel Core i5/i7 or equivalent, minimum 8 GB RAM; GPU for deep-learning training; 256 GB SSD or higher.
- Active internet connection is required for API integrations and MongoDB Atlas when used.

2.5 Design and Implementation Constraints

- **Dataset Constraint:** The ML model shall be trained and evaluated using the PlantVillage dataset and the crop/disease classes selected for the project.
- **Model Constraint:** The disease classification component shall use a **VGG16-based CNN with a custom Spatial Attention mechanism**.
- **Image Processing Constraint:** The system shall use OpenCV and TorchVision for image preprocessing, resizing, normalization, augmentation, and ROI-oriented processing.
- **Computational Constraint:** Deep-learning model training may require higher computational resources, including a GPU, while the deployed application should operate according to the available hardware.
- **Prediction Constraint:** The system shall use a confidence threshold to identify uncertain predictions and should not present low-confidence results as definite diagnoses.

- **RAG Constraint:** Decision-support information shall depend on the availability and quality of the curated agricultural knowledge base.
- **API Dependency:** The Gemini API requires an active internet connection and valid API configuration for AI-generated decision-support responses.
- **Database Constraint:** MongoDB or MongoDB Atlas shall be available for storing user information, scan history, advisor information, and appointment details.
- **Web Application Constraint:** The system shall be implemented using the specified technologies, including Flask, PyTorch, TorchVision, OpenCV, MongoDB, HTML, CSS, JavaScript, and ReportLab.
- **Professional Use Constraint:** The system is intended to provide decision-support information and shall not replace diagnosis or recommendations from qualified agricultural professionals.

2.6 Assumptions and Dependencies

- Users provide sufficiently clear crop leaf images.
- The trained model and class-label mapping are available to the application.
- The curated agricultural knowledge base contains relevant information for supported diseases.
- Gemini API credentials/configuration are available where required.
- MongoDB is accessible for application data storage.
- Internet connectivity is available for external API/database services.

3. SPECIFIC REQUIREMENTS

3.1 Data Requirements

3.1.1 Training and Testing Data

The primary dataset planned for training and testing is the PlantVillage Dataset. It contains labeled healthy and diseased plant-leaf images. The project uses the image labels corresponding to the crop/disease classes selected for the application.

Data Aspect	Requirement
Source	PlantVillage dataset
Data Type	Crop leaf images
Labels	Healthy/disease class labels in the selected subset
Preprocessing	Resize, normalization and ROI extraction
Training	VGG16 + spatial attention model training+MobileNetV2
Testing	Independent test images from the prepared dataset split

3.1.2 Data Processing

- Validate uploaded image type and readability.
- Resize images to the model input dimensions.
- Convert/process image colour information using OpenCV where required.
- Apply ROI-oriented preprocessing to reduce background interference.
- Normalize images using the model's preprocessing requirements.
- Apply suitable augmentation during model training.
- Maintain the class-label mapping consistently between training and inference.

3.1.3 Data Storage

- Store application/user and scan-history records in MongoDB.
- Store uploaded images according to the application's configured upload-storage mechanism.

- Protect model files, API credentials and database credentials from unauthorized access.
- Avoid storing unnecessary personal information.

3.2 Functional Requirements

User Authentication: The system shall allow authorized users to register, log in and access user-specific functionality.

Image Upload: The system shall allow a user to upload a crop leaf image through the web interface.

Image Validation: The system shall validate that the uploaded file is a supported and readable image.

Preprocessing: The system shall resize and normalize the input image and perform required OpenCV/TorchVision preprocessing.

ROI Extraction: The system shall perform ROI-oriented preprocessing to reduce irrelevant background information and focus on the leaf region.

ML Inference: The system shall pass the processed image to the VGG16-based CNN with spatial attention.

Classification: The system shall produce a predicted crop/disease class from the defined class set.

Confidence Score: The system shall provide a confidence score associated with the prediction.

Confidence Handling: The system shall identify low-confidence results using a configured confidence threshold and avoid presenting them as definite diagnoses.

RAG Retrieval: For a detected disease, the system shall retrieve relevant information from the curated agricultural knowledge base.

Decision Support: The system shall present relevant symptoms, precautions, preventive measures and management guidance.

AI Integration: The system shall use the configured Google Gemini API to support generation of disease-specific solutions using retrieved/contextual information.

PDF Report: The system shall generate a downloadable PDF report containing the detection result and relevant decision-support information.

Scan History: The system shall store and display relevant disease-scan history for the user.

Advisor Support: The system shall display available agricultural advisor information and expertise where configured.

Appointment Booking: The system shall allow users to select available advisor appointment slots and store booking information.

Error Handling: The system shall display understandable messages for invalid images, failed processing, unavailable services or other recoverable errors.

3.3 Performance and ML Quality Requirements

Evaluation Metrics:
The crop disease classification model shall be evaluated using **Accuracy, Precision, Recall, F1-Score, and RMSE** on the test dataset. Accuracy measures the overall correctness of disease classification, while Precision, Recall, and F1-Score evaluate the classification performance across disease classes. RMSE shall be included as an additional error-based evaluation metric as required by the SRS evaluation framework.

Minimum Quality Target:
The system shall target an overall classification **accuracy of at least 90%**, with Precision, Recall, and F1-Score

used to assess the reliability of disease classification. The target values shall be validated using an independent test dataset.

Prediction Confidence:
The system shall provide a confidence score with each predicted disease class. Predictions below the predefined confidence threshold shall be treated as uncertain rather than being presented as a definite disease diagnosis.

Inference Latency:
The system shall provide the ML prediction within an acceptable response time. The target inference latency shall be **less than 200 ms per image**, measured after image preprocessing and excluding image upload, RAG response generation, database operations, and PDF report generation.

Performance Monitoring:
The system shall record evaluation results including classification metrics and inference latency during model testing to verify that the implemented model satisfies the defined ML quality and performance requirements.

3.4 External Interface Requirements

3.4.1 User Interface

- Responsive web interface for image upload and result display.
- Dashboard/history view for previous scans.
- Result page showing predicted disease/class and confidence.
- Decision-support section showing retrieved guidance.
- PDF report download option.
- Advisor and appointment interface.
- Clear validation, loading and error messages.

3.4.2 Software Interfaces

- Flask backend interfaces with the web frontend.
- PyTorch/TorchVision interfaces with the trained VGG16 + spatial attention model.
- OpenCV performs image-processing operations.
- MongoDB provides persistent application data storage.
- Google Gemini API supports generative decision-support functionality.
- ReportLab generates PDF reports.

3.5 Security and Safety Requirements

- User authentication shall restrict access to user-specific records.
- Passwords shall be protected using secure password-hashing mechanisms.
- API keys and database credentials shall not be exposed in client-side code or public repositories.
- Model weights and training data shall be protected against unauthorized modification.
- Uploaded files shall be validated before processing.
- User scan history and appointment information shall be accessible only to authorized users/roles.
- The system shall clearly communicate that ML results are decision-support information and not a substitute for professional diagnosis.

3.6 ML Model Requirements

Requirement	Specification
Architecture	VGG16 backbone with custom spatial attention mechanism.

Input	Crop leaf image after preprocessing/ROI extraction.
Feature extraction	Hierarchical visual features such as edges, textures and shapes.
Attention	Spatial attention assigns greater importance to informative spatial regions.
Output	Defined crop/disease class and confidence score.
Training data	PlantVillage dataset and selected project classes.
Inference handling	Confidence threshold for uncertain predictions.
Evaluation	Accuracy, precision, recall, F1-score and confusion-matrix analysis.

3.7 RAG and Decision-Support Requirements

- The predicted disease shall be used as the retrieval context/query.
- The retrieval layer shall obtain relevant information from the curated agricultural knowledge base.
- Retrieved information shall be used to ground the generated decision-support response.
- The response should provide disease-specific symptoms, precautions, preventive measures and management guidance where information is available.
- The system may incorporate relevant user-provided agricultural context.
- The system should support multilingual recommendations as an enhancement where implemented.

4. APPENDICES

4.1 Glossary

Term	Meaning
CNN	Convolutional Neural Network used for image classification.
VGG16	Deep convolutional neural-network architecture used as the feature-extraction backbone.
ROI	Region of Interest; the relevant leaf region emphasized during preprocessing.
Spatial Attention	Mechanism that assigns higher importance to informative spatial regions.
RAG	Retrieval-Augmented Generation; retrieves knowledge before generating decision-support content.
LLM	Large Language Model used for generating natural-language responses.
PlantVillage	Plant-leaf image dataset used for crop disease classification.
Confidence Score	Model-associated value indicating confidence in a predicted class.
Gemini API	External generative-AI API used in the proposed decision-support workflow.
MongoDB	Database used for application records.
Flask	Python web framework used for the application backend.

4.2 Schema

Detection Collection:

Field	Data Type	Description
id	ObjectId	Unique detection record identifier
user_id	ObjectId	References the user who performed the detection

crop	String	Detected crop name
disease	String	Predicted disease name
confidence	Float	Prediction confidence percentage
image_path	String	Path of the uploaded crop image
timestamp	ISODate	Date and time of detection

User Collection:

Field	**Data Type**	**Description**
name	String	User/farmer name
email	String	User email address
password	Binary/Hash	Stored password hash
phone	String	User contact number
role	String	User role
district	String	User district
lat	Float	Latitude of user location
lng	Float	Longitude of user location
location	String	User's local location
preferred_language	String	Preferred language for communication
state	String	User state

Appointment Collection:

Field	**Data Type**	**Description**
farmer_id	ObjectId	References the farmer/user
advisor_id	ObjectId	References the agricultural advisor
date	String	Appointment date
time_slot	String	Selected appointment time
report_path	String	Path of the generated diagnosis PDF report
status	String	Appointment status, e.g. scheduled
created_at	ISODate	Appointment creation date and time

4.3 Future Enhancements

- Evaluate the system using field images under varied lighting, backgrounds and camera conditions.
- Add more crop and disease classes using validated labeled data.
- Improve the RAG pipeline with larger and more systematically curated agricultural knowledge bases.
- Introduce multilingual decision-support in regional Indian languages.
- Perform broader cross-dataset evaluation for robustness and generalization.
- Add additional explainability and model-monitoring features.